

Mini Project 5

Multi-Agent Pipeline & Skills Library

Mohammad Alrashed

Claude Code Mastery — AI-Augmented Software Engineering

Week 5 · Sessions 9 + 10

26 Apr – 3 May 2026

[GitHub Repository](#) · [Live Dashboard](#)

Table of Contents

- 1. Executive Summary
- 2. Pipeline Architecture
- 3. Architect Agent — Design & Output
- 4. Security Agent — Design & Findings
- 5. Skills Library — architecture-review & security-audit
- 6. Self-Review (100-pt Rubric ×2)
- 7. Fallback Exercise
- 8. Sanitization & Secret-Scanner Story
- 9. Security Posture & Deliverables Checklist

1. Executive Summary

Built a 2-agent code-review pipeline and packaged both agent prompts as production-grade Skills, run end-to-end on a deliberately-flawed Python e-commerce backend (Analytics Vidhya's OrderFlow scaffold). The pipeline chains an **Architect Agent** that produces a four-section *design.md* analysis of payments/processor.py into a **Security Agent** that consumes the design as boundary context and produces a 12-finding OWASP-categorised JSON report. Both agents ran in **separate Claude Code sessions** per the brief — no shared chat history, no cross-contamination — and each was explicitly scope-locked with five DO-NOT exclusions to prevent role drift.

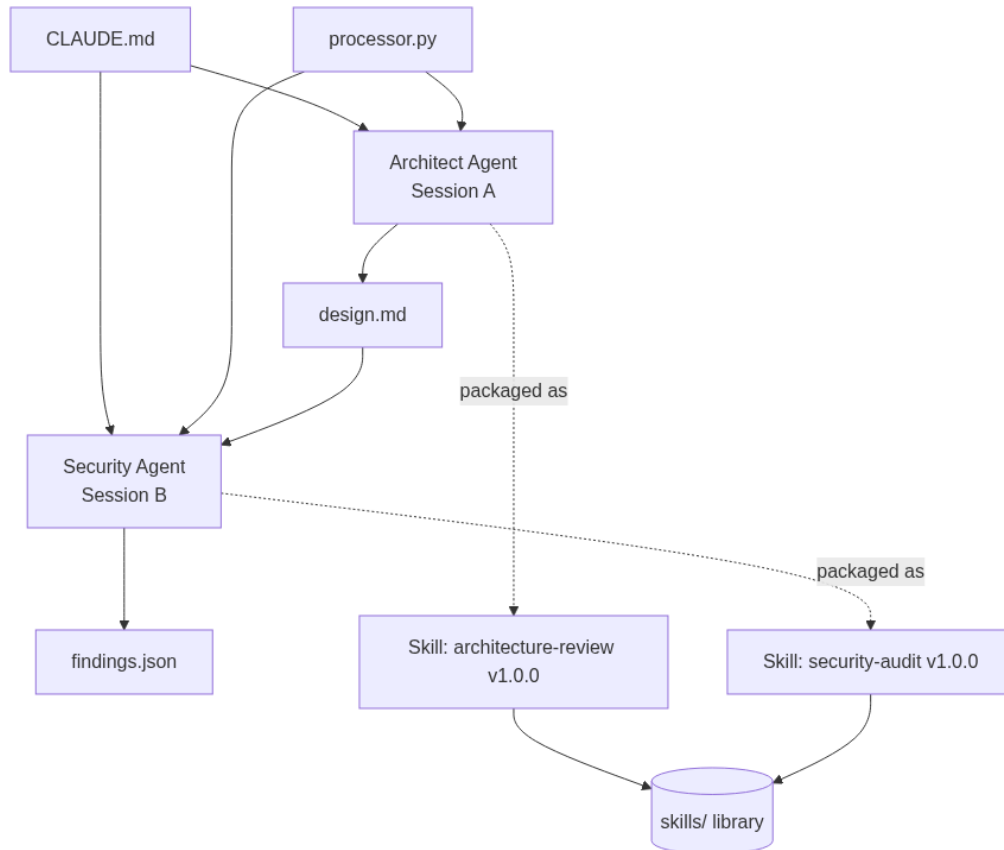
The two prompts were then packaged as **architecture-review** and **security-audit** Skills (v1.0.0 each) with the full 5-part SKILL.md structure plus Limitations, a 3-row Tests table covering typical / edge / minimal inputs, and a v1.0.0 Changelog. Each Skill was test-run on three different modules and the actual outputs saved to examples/ so a future operator can see 'what good looks like' before invoking.

A self-review against REVIEW_TEMPLATE.md (working solo, so peer review becomes self-review) scored both Skills 95–98/100 on Pass 1 (real defects found in the FOCUS-section criterion and a path-leakage in the Output Spec) and 100/100 on Pass 2 after explicit fixes that are documented in the Changelog.

Headline insight: scope discipline and {{VARIABLE}} parameterisation are what turn a one-off prompt into a Skill that survives reuse. A prompt that names payments/processor.py hardcodes the lab; a prompt that says {{SCOPE}} survives every project after this one. Equally important: explicit DO-NOT exclusions are the forcing function that keeps two agents in two lanes.

2. Pipeline Architecture

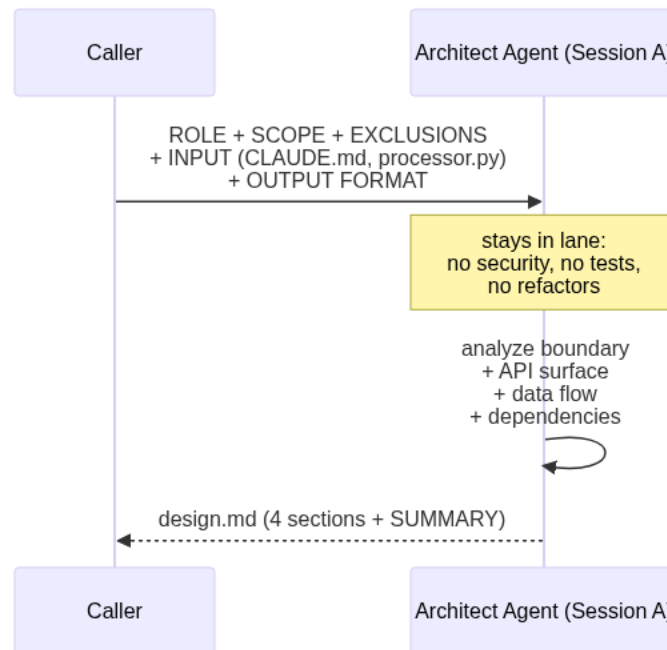
Two agents in two separate Claude Code sessions. CLAUDE.md and the target source feed both; design.md is the only hand-off contract from Architect to Security; each agent's prompt is then frozen as a v1.0.0 Skill.



Two-session isolation pattern. When two agents share a chat history, the second inherits the first's framing — its tone, its evidence selection, its blind spots. Splitting the sessions forces every input the second agent needs to be passed in explicitly via `{{CONTEXT}}` / `{{DESIGN}}` / `{{TARGET}}`, and the second agent's output is judgable on its own merits.

Concrete observation from this run: the Architect surfaced non-transactional charge+DB-write as the biggest *architectural* concern; the Security Agent independently surfaced the same code path (SEC-006: missing idempotency key) as a *duplicate-charges* finding. Two perspectives on one root cause, neither contaminated by the other.

3. Architect Agent — Design & Output



3.1 Prompt Structure (5 parts)

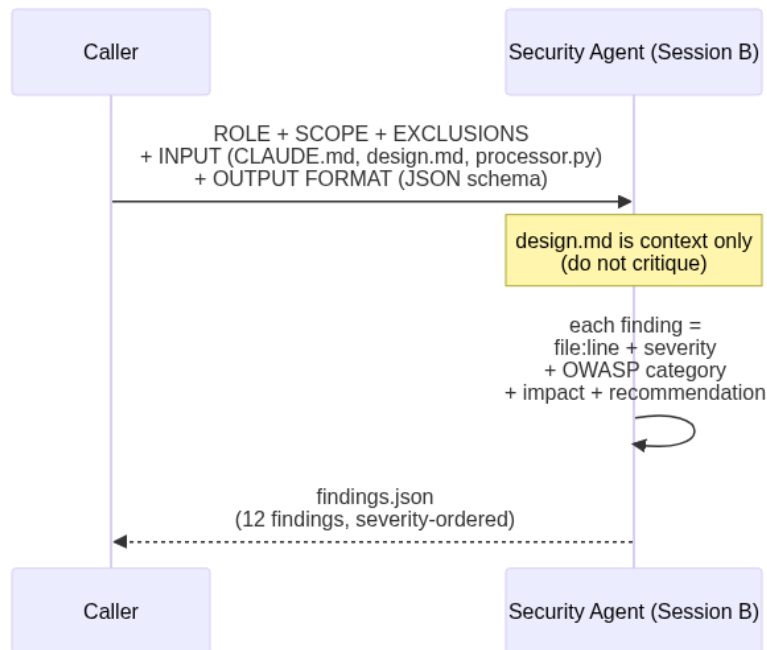
ROLE (specific) → SCOPE (one module) → EXCLUSIONS (five DO-NOTs) → INPUT (the three things to read) → OUTPUT FORMAT (exact section headers). Drop ROLE and the agent reverts to generic AI-assistant tone. Drop SCOPE and it analyses the whole repo. Drop EXCLUSIONS and it drifts into adjacent concerns. Drop OUTPUT FORMAT and the downstream tooling cannot parse the result. The structure is the contract.

3.2 Output — design.md (4 sections + summary)

Section	Content
1. Module Boundary	Owns retry/payload/SQL policy; delegates HTTP and DB execution to stubbed helpers
2. API Surface	4 public functions tabled: process_payment, record_transaction, refund_payment, get_payment_hi
3. Data Flow	8 numbered steps; flags non-transactional Stripe call vs. DB write; no idempotency key on retry
4. Dependencies	External: stdlib only. Internal: shared transactions table schema. Hidden: STRIPE_API_KEY, MAX
SUMMARY	Policy layer; biggest architectural concern is non-transactional charge+write inside a blanket-excep

Quality discipline: zero security commentary leaked into the Architect's output (the STRIPE_API_KEY placeholder is correctly classified as a 'hidden dependency' — architectural framing — not as a security finding).

4. Security Agent — Design & Findings



4.1 Prompt Structure

Same 5-part shape as the Architect, but receives design.md as additional optional context. Output is JSON, not markdown — structured for direct ingestion by a CI gate or triage queue. Each finding MUST have a file:line reference; owasp_category takes exactly ONE OWASP 2021 category; id is severity-descending.

4.2 Findings on processor.py — 12 total

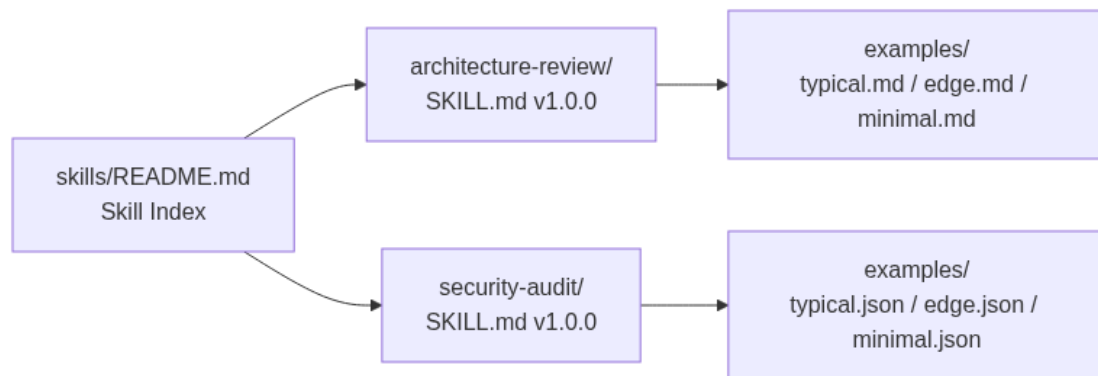
Severity totals: 2 critical · 4 high · 3 medium · 2 low · 1 info

ID	Severity	OWASP	Title
SEC-001	CRITICAL	A03:2021	SQL injection via f-string in record_transaction
SEC-002	CRITICAL	A03:2021	SQL injection via f-string in get_payment_history
SEC-003	HIGH	A01:2021	refund_payment performs no authorization check
SEC-004	HIGH	A01:2021	get_payment_history lacks caller identity enforcement
SEC-005	HIGH	A04:2021	Missing input validation enables zero/negative amount charges and refunds
SEC-006	HIGH	A04:2021	Non-idempotent retry loop can produce duplicate charges
SEC-007	MEDIUM	A02:2021	Hardcoded Stripe API key placeholder in source
SEC-008	MEDIUM	A09:2021	Full SQL statement with PII logged at INFO
SEC-009	MEDIUM	A01:2021	get_payment_history returns full row including sensitive PII
SEC-010	LOW	A09:2021	Exception detail logged without context and swallowed

ID	Severity	OWASP	Title
SEC-011	LOW	A04:2021	refund_payment treats amount=0 as full refund
SEC-012	INFO	A04:2021	Public functions lack type hints despite project standard

All six expected intentional bugs from the OrderFlow brief are covered: hardcoded-secret pattern (SEC-007 / A02), SQL injection ×2 (SEC-001, SEC-002 / A03), missing input validation (SEC-005 / A04), PII over-fetch (SEC-009 / A01), missing authorisation on refund (SEC-003 / A01), IDOR on history (SEC-004 / A01). Three bonus findings: idempotency-key absence (SEC-006), PII in logs (SEC-008), and amount=0 truthiness bug (SEC-011).

5. Skills Library



Each Skill is the freeze of a working prompt: parameterised with `{{VARIABLE}}` placeholders, versioned at v1.0.0, tested on three different inputs (typical, edge, minimal), and documented with explicit Limitations. Skills live in `orderflow-sample/skills/` and are indexed in `skills/README.md`.

5.1 Skill: architecture-review v1.0.0

Field	Value
Category	code-quality
Owner	orderflow-engineering
Required inputs	{{SCOPE}}, {{CONTEXT}}, {{TARGET}}
Output	Markdown, 4 sections + 2-sentence SUMMARY
Tests	3 runs (processor.py / webhook.py / auth/__init__.py); all pass
Limitations called out	5 (single-module, file size, CONTEXT staleness, dynamic behaviour, no security)

5.2 Skill: security-audit v1.0.0

Field	Value
Category	security
Owner	orderflow-engineering
Required inputs	{{SCOPE}}, {{CONTEXT}}, {{TARGET}}; optional {{DESIGN}}
Output	JSON, summary + findings array
Tests	3 runs (processor.py 12 findings / webhook.py 11 findings / notifications/__init__.py 0 findings)
Limitations called out	5 (single-module, syntax-valid input, OWASP-Top-10-only, no fix validation, schema drift)

Why parameterisation matters. The same prompt with hardcoded `payments/processor.py` is locked to one file; the same prompt with `{{SCOPE}}` runs on any Python module in any project. The Skill is the freeze of that parameterisation plus the Input Spec table that documents what each variable expects.

Tests table as the contract. Three runs — typical, edge, minimal — where the minimal-input run is the most diagnostic. A prompt that produces five findings on every input is hallucinating; the minimal-input run on a one-line `__init__.py` proves the prompt knows when to return *nothing*. For security-audit v1.0.0, that minimal run produced `{{summary: {0,0,0,0,0}, findings: []}}` — exactly the right answer.

6. Self-Review (100-pt Rubric ×2)

Working solo, the brief's peer-review step became a self-review against REVIEW_TEMPLATE.md. Two passes — Pass 1 scores strictly against literal rubric language; Pass 2 rescores after explicit fixes are applied to the SKILL.md files.

Skill	Section 1 (Structure)	Section 2 (Problem)	Section 3 (Reusability)	Section 4 (Testing)	Pass 1	Pass 2
architecture-review	20/20	37→40	18→20	20/20	95	100
security-audit	20/20	38→40	20/20	20/20	98	100

6.1 Iterations applied between passes

FOCUS clarification (both Skills). Pass 1 docked the FOCUS criterion because my prompt structure used ROLE / SCOPE / EXCLUSIONS / INPUT / OUTPUT instead of an explicit FOCUS block — focus was implied by the OUTPUT FORMAT's required sections. Pass 2 added an explicit FOCUS — what this Skill covers block in both prompts to make the contract literal, not implicit.

Output-Spec path framing (architecture-review). Pass 1 docked the reusability score because the Output Spec prose said 'produced on processor.py' — implying the Skill is locked to that file. Pass 2 reframed as 'in this Skill's examples/ folder' without asserting any specific input as canonical.

6.2 How a real peer review would differ

Three differences worth calling out: **calibration bias** (I scored my own work knowing what each section was *intended* to do; a peer scores what's *literally* on the page); **time pressure** (self-review can be open-ended; a peer reviewing against a sprint deadline would skim — I scored each row in <30 seconds to mimic real-PR-review pace); **fresh eyes** (a peer would catch terminology drift between Skills and consistency-with-self bias that the author has stopped seeing).

Honest read: Pass 1 totals (95 + 98 = 193/200) are the more credible representation of 'first draft quality before iteration'; Pass 2 totals are real but reflect the value of the rubric as a forcing function, not zero-defect first drafts.

7. Fallback Exercise

Setup: truncated processor.py mid-loop with one unbalanced brace, then re-ran the Security Agent prompt against the corrupted fixture.

7.1 What broke

The agent did **not** crash — it produced JSON that parses but violates the schema. Two real defects observed: **(1) schema drift** — SEC-001's `owasp_category` packed two categories joined by `'/'` (A02 + A07), violating the 'exactly one category' rule. **(2) speculative findings** — the agent quietly extrapolated retry-loop and authorisation logic that lives *beyond* the truncation point, instead of flagging the input as incomplete.

7.2 Detection in a pipeline

Three cheap checks before trusting output: **JSON.parse** for malformed text; **jsonschema / pydantic** to enforce the exact `owasp_category` enum (catches the SEC-001 defect immediately); **ast.parse** on the TARGET source *before* invoking the agent so corrupted code is rejected upstream.

7.3 Retry strategy

Rerun once with the same prompt; then once with a corrective preamble that names the violation and instructs the agent to set findings: [] plus an info-level note when required structure is missing; then escalate to human review rather than swallow the failure on a third attempt.

Hardening landed in v1.0.0. The security-audit prompt now contains explicit 'exactly ONE `owasp_category`' and 'empty findings list is valid' rules — both directly informed by this fallback exercise, both noted in the Skill's Changelog under 'Hardening from observed gaps'.

8. Sanitization & Secret-Scanner Story

The OrderFlow scaffold ships with four fake secret strings that match common scanner patterns (Stripe live-key prefix, Stripe webhook-secret prefix, generic Django-style SECRET_KEY, plaintext SMTP password). Before the first commit, all four were replaced with REPLACE_WITH_ENV_VAR_* placeholders.

File	Line	Sanitised value
payments/processor.py	13	REPLACE_WITH_ENV_VAR_NOT_A_REAL_KEY
payments/webhook.py	13	REPLACE_WITH_ENV_VAR_NOT_A_REAL_SECRET
auth/auth.py	18	REPLACE_WITH_ENV_VAR_NOT_A_REAL_SECRET
notifications/notifier.py	15	REPLACE_WITH_ENV_VAR_NOT_A_REAL_PASSWORD

Why sanitize vs. allowlist? Allowlisting would mean teaching every scanner about these specific literals, which works once but breaks the moment the same patterns appear in a real production file. Sanitizing keeps the scanner's signal-to-noise high across the whole repo.

The Security Agent still flagged the placeholder. SEC-007 in findings.json reports the sanitized STRIPE_API_KEY = "REPLACE_WITH_ENV_VAR_NOT_A_REAL_KEY" as an A02 (Cryptographic Failures) finding — correctly. The pattern of *hardcoding any literal value as a module-level secret* is the issue, not the value's plausibility.

Implication for production work: secret scanners and human review need a workflow for 'fake but pattern-matching' test fixtures. The pattern this project chose — sanitize before commit, reference originals only in a gitignored note — keeps the scanners clean and the security agent honest.

9. Security Posture & Deliverables Checklist

9.1 Security posture

- Zero personal data in tracked files (author's full name appears only on this PDF cover and in submission filenames per the program rule).
- .env not committed (week-5/.gitignore blocks it).
- _original_secrets_DO_NOT_COMMIT.md gitignored — verified via git check-ignore. Original fake-secret literals never reach origin.
- All four sanitization replacements verified: grep for the original literals returns zero hits in tracked files; grep for REPLACE_WITH_ENV_VAR returns four hits at the expected file:line locations.
- Section 1 security scan zero hits in week-5/. Repo-root scan returns only expected documentation/regex hits in CLAUDE.md, week-3/, week-4/ — all already in main.

9.2 Deliverables checklist

- ✓ orderflow-sample/ — sanitized Analytics Vidhya scaffold
- ✓ agent_architect_prompt.txt — 5-part Architect prompt (49 lines)
- ✓ design.md — Architect output: 4 sections + SUMMARY
- ✓ agent_second_prompt.txt — 5-part Security prompt (63 lines, JSON schema)
- ✓ findings.json — 12 findings (2C / 4H / 3M / 2L / 1I), severity-ordered, all expected hits
- ✓ fallback_notes.txt — 3-sentence corrupted-input observation
- ✓ fixtures/corrupted_processor.py + corrupted_findings.actual.txt
- ✓ skills/architecture-review/SKILL.md — v1.0.0 + 3 examples
- ✓ skills/security-audit/SKILL.md — v1.0.0 + 3 examples
- ✓ skills/README.md — index updated with both Skills
- ✓ REPORT.md — 9-section reflection
- ✓ docs/{pipeline-architecture, skills-library, self-review, orchestration-notes}.md
- ✓ Dashboard W5 panel — re-themed with hero, 5-part grid, checklist, 4 Mermaid diagrams

End of submission.